

MILOS CVETKOVIC

Senior Full Stack Software Engineer

DETAILS

ADDRESS

Belgrade
Serbia

PHONE

+38162238809

EMAIL

cvetkovicmilos@gmail.com

LINKS

[LinkedIn](#)

[Twitter](#)

SKILLS

Claude Code

NodeJS

BunJS

JavaScript

TypeScript

NestJS

Elysia

PROFILE

Results-driven JavaScript Tech Lead with 10+ years turning legacy codebases into cloud-native solutions. Led full-stack modernization of a trading platform: redesigned architecture following Clean Architecture and DDD principles, built multi-environment Terraform infrastructure on Azure, automated CI/CD pipelines with security scanning, and pioneered AI-assisted development with Claude Code. Built a self-healing agent using Claude Agent SDK for autonomous production error resolution. Passionate about clean code, automation, and exceptional developer experience. Expert in TypeScript, React, NestJS, PostgreSQL, Azure, Kubernetes, and modern DevOps practices.

EMPLOYMENT HISTORY

Senior Full Stack Engineer, Obsidian 22

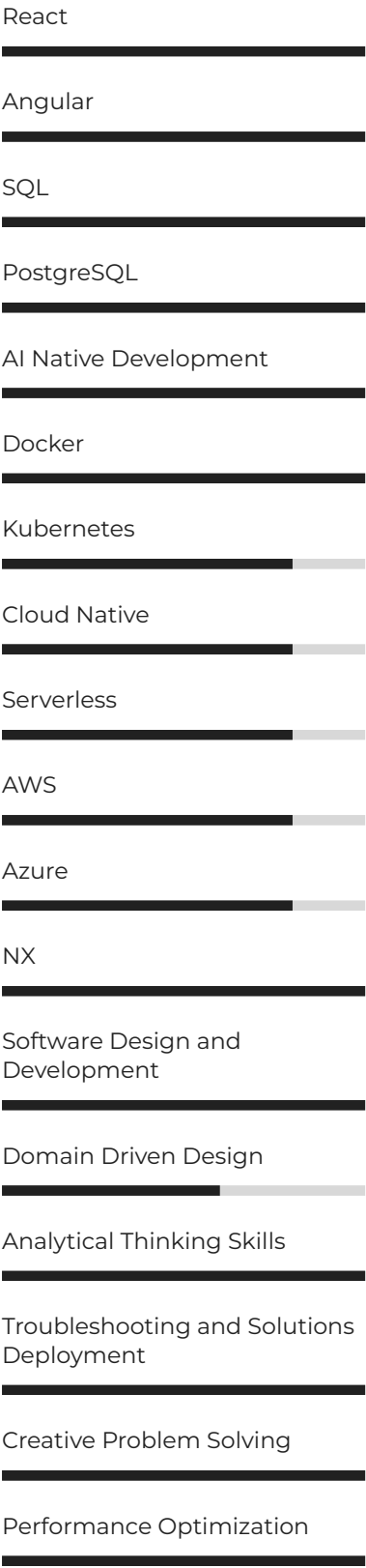
Dec 2025 — Present

Lead developer on a comprehensive commodity trading platform modernization initiative. Responsible for full-stack development,

infrastructure automation, CI/CD architecture, and pioneering AI-assisted development workflows using Claude Code.

Technical Stack:

- **Frontend:** React 18, Material-UI (MUI), Vite, TypeScript, React Router
- **Backend:** Express.js/Node.js, TypeORM, PostgreSQL, Zod validation, pg-boss (background jobs)
- **Testing:** Jest (unit/integration), Playwright (E2E with 4-way sharding), Supertest
- **Infrastructure:** Microsoft Azure (Container Apps, PostgreSQL Flexible Server, Communication Services, Front Door, Log Analytics)
- **IaC:** Terraform with multi-environment architecture (development, production, shared)
- **CI/CD:** GitHub Actions, Nx-powered monorepo with intelligent caching and affected-only builds
- **AI/Automation:** Claude Code for AI-assisted development, Claude Agent SDK (Anthropic) for self-healing agent



LANGUAGES



- **High-Performance Services:** Bun runtime + Elysia framework
- **Integration:** Sage 200 accounting system (OAuth, real-time sync)

Key Contributions (308+ commits):

Feature Development:

- Implemented server-side pagination and filtering for high-volume stock management
- Built background job system for async email confirmations using pg-boss
- Developed comprehensive data seeding library with configurable generation profiles
- Created PDF generation system for invoices and confirmations

CI/CD & DevOps:

- Architected modular CI/CD pipeline using reusable GitHub composite actions
- Implemented security scanning (Gitleaks, npm audit, Trivy container scanning)
- Built Nx-based affected-only deployment strategy reducing CI time significantly
- Created health-check-retry and docker-build-push reusable workflows

Infrastructure:

- Designed multi-environment Terraform architecture with proper state isolation
- Implemented GitHub OIDC authentication for secure Azure deployments
- Configured Azure Front Door with custom domains and SSL certificates
- Set up Azure Communication Services for transactional emails

Testing & Quality:

- Established Playwright E2E testing framework with Page Object pattern
- Implemented 4-way test sharding for parallel CI execution
- Created comprehensive test fixtures and factory patterns

AI Innovation:

- Pioneered use of Claude Code for AI-assisted development across the entire codebase
- Designed a self-healing agent using Claude Agent SDK that:
- Monitors production errors via Azure Log Analytics
- Automatically analyzes and proposes fixes using AI
- Creates PRs with proper validation and human approval gates
- Authored organization-wide technology strategy documentation

Architecture:

- Applied Clean Architecture and DDD principles throughout the codebase
- Created shared domain-types library for cross-application type safety
- Enforced architectural boundaries using Nx module boundary rules

Business Impact:

- Modernizing legacy commodity trading operations with real-time Sage 200 integration
- Supporting deals, sales, purchases, invoicing, stock management, and customer workflows
- Enabling faster development cycles through comprehensive automation and AI assistance
- Reducing production incidents through self-healing capabilities and robust CI/CD

Notable Practices:

- Trunk-based development with short-lived feature branches
- Comprehensive documentation (ADRs, runbooks, architecture docs)
- Using Claude Code as primary development companion for code generation, refactoring, and debugging

JavaScript Tech Lead, Levi9

Belgrade

Sept 2021 — Nov 2025

- Lead cross-functional teams in the development of scalable web applications.
- Design and implement a microservices architecture to enhance system performance.
- Drive technical strategy for enterprise-level projects, ensuring alignment with business goals.
- Optimize application performance using cloud-native solutions on AWS and Azure.
- Mentor junior developers, fostering growth and improving team collaboration.

Lead FrontEnd Engineer, Quantlabs doo

Belgrade

Jan 2021 — Sept 2021

- Led cross-functional teams in developing scalable web applications using the React framework with the NX monorepo.
- Executed performance optimization strategies, improving application efficiency and user experience.
- Drove technical strategy for enterprise-level projects, ensuring alignment with business objectives.
- Collaborated on architectural decision records (ADRs) to inform technical strategy in platform modernization.

JavaScript Tech Lead, Levi9

Belgrade

Apr 2016 — Jan 2021

- Led cross-functional teams to design and implement scalable web applications, enhancing performance and user experience.
- Drove technical strategy for enterprise-level projects, ensuring alignment with business goals and industry standards.
- Developed and maintained a microservices architecture, promoting efficient and modular software design.
- Spearheaded cloud-native solutions utilizing AWS and Azure, optimizing deployment processes.
- Contributed to architectural decision records (ADRs), guiding modernization initiatives and improving software architecture.

JavaScript Engineer, TNation

Belgrade

May 2015 — Apr 2016

FrontEnd Developer, Comtrade

Mar 2014 — May 2015

FrontEnd Developer, Razor Communications

Feb 2013 — Mar 2014

CERTIFICATIONS

Angular Certified Architect

a program from angulararchitects.io, created by a famous Angular trainer [Manfred Steyer](#)

PROJECTS

Commodity Trading System

Dec 2025 — Present

A comprehensive Nx monorepo powering a commodity trading platform modernization. The platform manages the complete trading lifecycle

including deals, sales, purchases, invoicing, stock management, customer relationships, and accounting integration with Sage 200.

Monorepo Architecture: Single package.json, trunk-based development, affected-only CI/CD builds

Applications

1. Backend API Service

Domain: Core Business Logic & Data Management

LEGACY CODEBASE - Inherited Express.js application undergoing incremental modernization

Tech Stack:

- Runtime: Node.js with TypeScript
- Framework: Express.js (planned migration to NestJS)
- Database: PostgreSQL with TypeORM ORM
- Validation: Zod schemas
- Background Jobs: pg-boss (PostgreSQL-backed queue)
- PDF Generation: pdfmake
- Email Templating: Pug

What It Does:

- RESTful API serving all business operations
- Manages entities: Customers, Deals, Sales, Purchases, Invoices, Stock, Products, Credit Notes
- Handles multi-currency operations with exchange rate management
- Integrates with Sage 200 accounting system (OAuth-based sync)
- Processes background jobs (email confirmations, data sync)
- Generates PDFs for invoices and confirmations

Modernization Efforts:

- Architecture: Migrated from flat file structure with mixed concerns to Clean Architecture layers (controllers → services → repositories)
- Validation: Replaced manual inconsistent validation with Zod schemas and type inference
- Error Handling: Replaced generic try/catch with custom error classes and proper HTTP mapping
- Background Jobs: Moved synchronous blocking operations to pg-boss async job queue for emails and heavy operations
- Database: Replaced raw SQL queries with TypeORM entities and versioned migrations
- Type Safety: Eliminated loose typing and any usage, implemented strict TypeScript with shared domain types library
- Testing: Added unit and integration tests with proper mocking where there was minimal coverage
- API Design: Standardized inconsistent endpoints to RESTful conventions with pagination and filtering
- Framework: Planning NestJS migration for Q2-Q3 2026 to replace minimal Express.js structure

Business Value:

- Central source of truth for all trading operations
- Automates invoice posting to accounting system
- Reduces manual data entry through Sage integration
- Enables real-time stock visibility and management

2. Frontend Web Application

Domain: User Interface & Experience

LEGACY CODEBASE - Inherited React application with ongoing UX and architecture improvements

Tech Stack:

- Framework: React 18 with TypeScript
- UI Library: Material-UI (MUI) v5
- Build Tool: Vite
- Routing: React Router v6
- State: React Context + custom hooks
- HTTP: Axios with custom API hooks

What It Does:

- Single-page application for trading operations
- Domain-organized components (customers, deals, invoices, sales, purchases, stock)
- Paginated data tables with server-side filtering
- Form management with validation
- Toast notifications for user feedback
- Responsive design for desktop users

Modernization Efforts:

- Build System: Migrated from Webpack (slow builds) to Vite (10x faster HMR)
- Component Structure: Refactored monolithic components into domain-organized, composable components
- API Layer: Consolidated scattered fetch calls into centralized API hooks (useCustomersApi, useSalesApi)
- Type Safety: Replaced implicit any and loose types with strict TypeScript and shared domain types
- Data Fetching: Implemented server-side pagination and filtering where only client-side pagination existed
- State Management: Eliminated prop drilling with React Context providers and custom hooks
- Testing: Added Jest and React Testing Library tests where there were none
- Error Boundaries: Implemented React Error Boundary for graceful failure handling

- Forms: Converted uncontrolled forms with manual validation to controlled forms with validation helpers

Business Value:

- Intuitive interface for daily trading operations
- Real-time data visibility across all business entities
- Streamlined workflow for invoice approval and posting
- Reduced training time through consistent UX patterns

3. E2E Test Suite

Domain: Quality Assurance & Regression Testing

Tech Stack:

- Framework: Playwright
- Pattern: Page Object Model
- Fixtures: Custom auth and test data fixtures
- CI: 4-way sharded parallel execution

What It Does:

- Comprehensive end-to-end testing across all user journeys
- Authentication flow testing (login, session persistence)
- CRUD operations for all major entities
- Sage integration testing (posting, sync, error scenarios)
- Mock API support for isolated testing
- Tagged test organization (@smoke, @sage, @crud)

Business Value:

- Catches regressions before production deployment
- Validates critical business workflows automatically
- Reduces QA cycle time through parallelization
- Documents expected system behavior through executable specs

4. Build Cache Server

Domain: Developer Infrastructure & CI Optimization

Tech Stack:

- Runtime: Bun (high-performance JavaScript runtime)
- Framework: Elysia (fast web framework)
- Storage: Azure Blob Storage + LRU in-memory cache
- Auth: Dual-token (read/write) with timing-safe comparison

What It Does:

- Remote cache server for Nx build artifacts
- LRU in-memory caching for hot artifacts
- Azure Blob Storage for persistent cache
- Rate limiting (1000 req/min)
- Health checks for container orchestration

Business Value:

- Dramatically reduces CI/CD build times (cache hits skip rebuilds)
- Enables consistent builds across developer machines
- Reduces cloud compute costs through build reuse
- Improves developer productivity with faster feedback loops

5. Self-Healing Agent

Domain: AI-Powered Operations & Incident Response

Tech Stack:

- Runtime: Bun
- Framework: Elysia
- AI: Claude Agent SDK (Anthropic)
- Monitoring: Azure Log Analytics
- Storage: Azure Table Storage
- VCS: GitHub API

What It Does:

- Monitors production errors via Azure Log Analytics webhooks
- Uses Claude AI to analyze errors and propose fixes
- Automatically creates pull requests with fixes
- Monitors CI pipeline and retries on failure (max 3 attempts)
- Implements safety constraints (daily limits, budget caps, confidence thresholds)
- Provides kill switch and emergency override controls
- Tracks learning metrics for calibration improvement

Business Value:

- Reduces mean time to resolution (MTTR) for production incidents
- Frees developers from repetitive bug fixes
- Provides 24/7 automated incident response
- Builds institutional knowledge through fix pattern learning

- Maintains human oversight with approval gates

Shared Libraries

Domain Types Library

Purpose: Cross-application type safety and DDD building blocks

Contents:

- Value objects (Money, Quantity, DateRange, Address)
- Domain interfaces (IDataGenerator, IRepository)
- Custom error classes
- Domain services

Business Value:

- Ensures type consistency across frontend and backend
 - Prevents duplicate type definitions
- Enforces domain-driven design patterns

Data Seeding Library

Purpose: Test data generation for development and testing

Contents:

- Configurable data generators (fast, standard, large profiles)
- Factory patterns for all entities
- Realistic fake data with proper relationships

Business Value:

- Enables rapid local development with realistic data
- Supports load testing with large dataset generation
- Maintains data consistency for integration tests

Infrastructure (Terraform)

LEGACY CODEBASE - Inherited ad-hoc Azure resources migrated to Infrastructure as Code

Environments: Development, Production, Shared

Azure Resources:

- Container Apps (backend, frontend, cache server)
- PostgreSQL Flexible Server
- Front Door (CDN, SSL, custom domains)
- Communication Services (transactional email)
- Log Analytics & Application Insights
- Blob Storage (file uploads, cache)
- Key Vault (secrets management)

Modernization Efforts:

- Provisioning: Replaced manual Azure Portal clicks with Terraform Infrastructure as Code
- State Management: Implemented remote state in Azure Storage with locking where there was no tracking
- Environments: Created multi-environment setup (dev, production, shared) from single environment with no isolation
- Secrets: Migrated hardcoded scattered secrets to Azure Key Vault with OIDC authentication
- Networking: Configured Front Door CDN with custom domains and SSL replacing default configurations
- CI/CD Auth: Replaced service principal secrets with GitHub OIDC (secretless authentication)
- Deployments: Automated manual deployments with GitHub Actions and affected-only deploys
- Database: Upgraded basic tier to Flexible Server with geo-redundant backups
- Monitoring: Added Log Analytics and Application Insights where there was none
- Security: Implemented Trivy container scanning and Gitleaks secrets detection
- Documentation: Created runbooks, ADRs, and architecture docs where there was none

CI/CD Modernization:

- ci.yml: Build and test with Nx affected-only, parallel jobs, and caching
- deploy.yml: Deploy apps to Container Apps with health checks and rollback support
- terraform-validate.yml: IaC validation with format, validate, and plan on PRs
- terraform-deploy.yml: IaC apply with manual trigger and environment gates
- security-scan.yml: Security scanning with Gitleaks, npm audit, and Trivy weekly scans

Composite Actions Created:

- setup-node-npm: Consistent Node.js setup with caching
- health-check-retry: HTTP health checks with exponential backoff
- docker-build-push: Standardized container builds to GHCR
- terraform-setup: Terraform with Azure OIDC authentication

- container-app-deploy: Azure Container App deployment with rollback

Business Value:

- Infrastructure as Code ensures reproducible deployments
- Multi-environment isolation for safe testing
- Auto-scaling container apps handle load spikes
- Centralized monitoring for operational visibility
- Security scanning prevents vulnerabilities reaching production

Solar Energy Production Application

Levi9

2024 — 2025

Project Description: Application for Solar Energy Production working in primary project process

Project Role: JavaScript Tech Lead/Software Architect Responsibilities: System Architecture

Tools & Technologies: NodeJS, Azure, Serverless, Modular Monolith, Docker, AI Native Software Development

ECommerce Domain

Levi9

2022 — 2024

Project Description: Ecommerce project, payment processing platform Project Role: JavaScript

Tech Lead Responsibilities: FE App Architecture

Tools & Technologies: NextJS, TypeScript, MicroFrontEnd, AWS, Docker

Big Data Application

Levi9

2021 — 2022

Project Description: Big data project, resolving a problem with duplicated data Project Role: JavaScript

Tech Lead Responsibilities: FE App Architecture

Tools & Technologies: Angular, NgRx, NX, Reactive Architecture, AWS, Docker

Fintech Application

Quantlabs

2021 — 2021

Project Description: Fintech project, portfolio management app

Project Role: Lead FE Engineer Responsibilities: FE App Architecture

Tools & Technologies: React, Redux, RxJs, TS, NX, Jenkins

ECommerce Domain

Belgrade

2020 — 2021

Project Description: Shop for Linda portal

Project Role: JavaScript Tech Lead Responsibilities: FE App Architecture

Tools & Technologies: NextJs, Redux, TS, Docker, Terraform, AWS, Jenkins, Jest

ECommerce

Levi9

2019 — 2020

Project Description: Shop for group buy

Project Role: JavaScript Tech Lead Responsibilities: FE App Architecture

Tools & Technologies: NextJs, Redux, TS, Docker, Terraform, AWS, Jenkins, Jest

Voetbal TV App

Levi9

2017 — 2019

Project Description: Social application for amateur football fans

Project Role: Senior JavaScript Engineer

Tools & Technologies: Hybrid mobile application, greenfield project

Tech stack:

- Ionic engine
- Angular 2-4 versions
- TypeScript
- RxJS
- NGRX
- Redux
- NodeJS
- ExpressJS
- Serverless framework
- AWS - Lambda, DynamoDB

Media Domain Application

Levi9

2016 — 2017

Project Role: Medior JavaScript Engineer

Legacy application

Tech Stack:

- Hybrid mobile app - Cordova engine
- TypeScript 0.9 (version)
- KnockoutJS
- Custom-made third-party library for routing